

AI Data Analytics Agent: Technical Architecture & System Workflows Document

1. Executive Summary & System Vision

The **AI Data Analytics Agent** is an enterprise-grade, high-performance Business Intelligence (BI) and automated data analytics platform. It bridges the gap between raw, unstructured/structured data and decision-ready, executive insights. Operating on a decoupled client-server architecture, it processes tabular files (CSV, Excel, TSV, Parquet) and textual documents (PDF, Word, Plaintext), performs automated diagnostic data audits, runs interactive data cleaning cycles, compiles dynamic multi-agent SQL queries, aggregates and visualizes datasets on the fly using Plotly, handles semantic search and Q&A; via Retrieval-Augmented Generation (RAG), schedules background report compilation, and exports executive deliverables in multiple formats (PDF, PowerPoint, Word, Excel).

This document serves as the complete technical manual outlining the architecture, the inner workings of its multi-agent systems, end-to-end execution flows, API dictionaries, database schemas, and configuration requirements.

2. Decoupled System Architecture & Technology Stack

The platform uses a modular, decoupled structure to separate client-side interactive rendering from server-side numerical computing and LLM orchestration.

```
graph TB
    subgraph Frontend Console
        React[React 19 / Vite 8 Client Console]
        UI[App.jsx Main Page]
        UP[Upload Panel]
        CP[Chat Panel]
        CT[Chart View Plotly]
        WT[Result Table]
        WS[Workspace Sidebar]
        AX[Axios API Client]
    end
    subgraph Backend Server
        FastAPI[FastAPI / Uvicorn Server on Port 8000]
        MA[Main Entry: backend/main.py]
        subgraph REST_API_Routers
            R1[upload_routes.py]
            R2[cleaning_routes.py]
            R3[chat_routes.py]
            R4[export_routes.py]
            R5[scheduler_routes.py]
            R6[workspace_routes.py]
        end
        subgraph Multi-Agent_Systems
            OR[Orchestrator Agent]
            QA[Quality Agent]
            CL[Cleaning Agent / DataCleaner]
            SD[Schema & Metadata Agent]
            CG[CodeGenerationAgent]
            IA[InsightAgent]
            VA[VisualizationAgent / ChartRecommender]
            RAG[RAGAgent / Similarity Search]
            SCH[SchedulerAgent / APScheduler]
        end
        subgraph Storage_Analytics_Core
            SL[SQLite3 database: app.db]
            DK[DuckDB OLAP Engine]
            FA[FAISS Vector Store / SentenceTransformers]
        end
    end
    UP -->|Uploads Files| R1
    R1 -->|Triggers Cleaning Actions| R2
    R2 -->|Sends Questions| R3
    R3 -->|Calls Endpoints| REST
    REST -->|Reads files via Factory| CL
    CL -->|Saves cleaned versions| SL
    SL -->|Saves Dataframes| DK
    DK -->|Routes Query| OR
    OR -->|Quantitative SQL| CG
    CG -->|Qualitative Text| RAG
    RAG -->|Executes SELECT| DK
    DK -->|Queries Dense Embeddings| FA
    FA -->|SQL Dataframe| VA
    VA -->|SQL Result Sets| IA
    IA -->|Plotly Configuration| CT
    CT -->|Synthesized Insights| CP
    CP -->|Compiles PDF/Word/PPTX| SCH
    SCH -->|APScheduler Crons| DK
    DK -->|Saves recurring excel exports| SL
```

The Stack:

- **Backend Server:** FastAPI, Uvicorn, SQLite3, DuckDB, Pandas, NumPy, Sentence Transformers (**all-MiniLM-L6-v2**), FAISS, APScheduler, ReportLab, python-docx, python-pptx, and Google Gemini API (**gemini-2.5-flash**).
- **Frontend Client:** React 19, Vite 8, Tailwind CSS 4, Plotly.js, Framer Motion, Lucide React, and Axios.

3. Storage & Analytics Core Engine

At the heart of the backend are three database and retrieval engines configured for different workloads:

3.1 Metadata & Workspace Repository (SQLite3)

Managed by [sqlite_manager.py](https://github.com/ansible/ansible/blob/main/lib/ansible/modules/sqlite_manager.py), SQLite is used for persistent system configurations, session variables, user history, dataset versions, report catalogs, and cron schedules. On server boot, **SQLiteManager** sets up and migrates six database tables (detailed in Section 10).

3.2 High-Performance OLAP Engine (DuckDB)

Managed by [duckdb_manager.py](#), DuckDB functions as our in-memory, high-performance analytical engine. It handles analytical queries directly on tabular uploads. Features include:

- **Safe Dataframe Persistence:** Pandas dataframes are mapped to DuckDB tables using register operations or temporary file streams.
- **Write-Block Protection:** A query validator rejects queries containing forbidden keywords (**DELETE**, **DROP**, **ALTER**, **UPDATE**, **INSERT**) to enforce read-only execution.

3.3 Semantic Vector Database (FAISS)

Managed by [rag_agent.py](#), this component processes textual documents. It chunks documents and stores them in local index configurations using Facebook AI Similarity Search (FAISS) and Sentence Transformers, enabling fast k-Nearest Neighbor searches.

4. Multi-Agent System Deep-Dive

Rather than a single monolithic controller, the platform uses a multi-agent framework to isolate domains:

[illegible]

```
History ■■■■■ Small Match ■■ Adapt SQL ■■■■■ No ■■ Check Document RAG Workspace
■■■■ Run FAISS Search & Gemini ■■■■ No Cache Hit (Normal SQL Pipeline) ■■■■■
CodeGenerationAgent ■■ (Translates NL to DuckDB SQL) ■■■■■ DuckDB OLAP Engine ■■ (Query
Execution) ■■■■■ Data Integrity & Footnotes ■■ (Rule 1, 2, 3, 5 & Null Profiling) ■■■
■■ InsightAgent ■■ (Synthesizes findings to NL) ■■■■■ VisualizationAgent ■■ (Creates
interactive Plotly charts) ■■■■■ Consistency Validator ■■ (Checks LLM Alignment Audit) ■■
```

4.1 Ingestion Router & Parser Factory

Operates inside [parser_factory.py](#) to route uploads to specific loaders:

- **Tabular Loaders:** CSV (**csv_loader.py**), TSV (**tsv_loader.py**), Excel (**excel_loader.py**), and Parquet (**parquet_loader.py**).
- **Text/Document Loaders:** PDF (**pdf_loader.py** with pdfplumber), Word (**docx_loader.py**), and Plaintext (**txt_loader.py**).

4.2 Quality Profiler Agent

Defined in `quality_agent.py`, this agent runs diagnostic checks on the active dataset to identify:

- **Exact Duplicates:** Rows matching other rows exactly.
- **Near-Duplicates:** Record pairs matching by 90%+ similarity (using Levenshtein distance) in non-identifier columns.
- **Outliers:** Numerical elements lying outside IQR (Interquartile Range) or Standard Deviation boundaries.
- **Impossible Values:** Negative values for metrics like pricing, quantities, or age bounds (e.g., age > 120).
- **Casing Inconsistencies:** Variations in capitalization (e.g., "Chicago" vs "chicago").
- **Invalid Formats:** Currency indicators, percentage markers, or unparseable dates.

4.3 Data Cleaning Agent

Operating inside [cleaner.py](#), this agent applies structural fixes:

- **Data Imputation:** Fills null values with the median for metrics or labels them "Unknown" for dimensions.
- **Spell Alignment:** Consolidates spelling variations.
- **Derived Metrics Recovery:** Calculates missing values using mathematical logic:
 - $\text{revenue} = \text{units_sold} \times \text{unit_price}$
 - $\text{units_sold} = \text{revenue} \div \text{unit_price}$
 - $\text{unit_price} = \text{revenue} \div \text{units_sold}$
 - $\text{profit} = \text{revenue} - \text{cogs}$
 - $\text{cogs} = \text{revenue} - \text{profit}$
 - $\text{margin} = \text{profit} \div \text{revenue}$
- **Rollback Engine:** Tracks and saves clean dataframes in SQLite, allowing version rollback if changes impact calculations.

4.4 Code Generation Agent (NL-to-SQL)

Implemented in [code_agent.py](#), this agent converts natural language queries into DuckDB-compatible SQL statements:

- **LLM Prompts:** Prompts are dynamically compiled using dataset schemas and strict rules.
- **SQL Generation Prompt:** Contains guidelines to generate read-only statements, handle division-by-zero errors, format percentages, and exclude unassigned categories from rankings.
- **Rule-Based SQL Fallback:** A rule-based parser generates basic queries (sums, aggregations, limits) if the LLM API is unavailable.

4.5 Business Insight Agent

Defined in [insight_agent.py](#), this agent translates SQL result sets into plain-text summaries:

- **LLM Prompt Guidelines:** Prompts instruct the LLM to write factual answers based only on the query results.
- **Deterministic Fallback:** If the LLM is offline, a fallback summarizer calculates averages and categorical frequencies directly from the SQL output.

4.6 Visualization Agent & Chart Recommender

Located in [visualization_agent.py](#) and [chart_recommender.py](#), this agent selects the appropriate chart type:

1. **Date + Metric:** Line or Area chart.
2. **Category + Metric:** Bar or Pie chart.
3. **Two Numeric Fields:** Scatter Plot.
4. **Single Numeric Distribution:** Histogram or Box Plot.
5. **Multi-Dimensional Correlations:** Heatmap.

It generates JSON configuration objects for rendering interactive Plotly charts in the React UI.

4.7 RAG Semantic Agent

Defined in [rag_agent.py](#), this agent manages unstructured document text:

- **Indexing:** Chunks text by paragraph and indexes them in a local FAISS vector store.
- **Retrieval:** Computes embeddings for user queries and runs similarity searches.
- **Synthesis:** Compiles document excerpts and generates answers with source citations (e.g., **[Block X]**).

4.8 Cron Scheduler Agent

Located in [scheduler_agent.py](#), this agent manages background jobs via APScheduler:

- Supports daily, weekly, monthly, or cron trigger schedules.
- Runs query statements, exports results as Excel spreadsheets, and saves the logs to SQLite.

5. System Workflow Tracing

5.1 Workspace Upload & Ingestion

[User File Upload] ■■■> [upload_routes.py] ■■■> [ParserFactory] ■ [Determine File Type] ■■■■
Tabular (CSV/XLSX/Parquet) ■ ■■■■ Load Dataframe via Pandas ■ ■■■■ Build Schema & Profile
Metadata ■ ■■■■ Create Workspace in SQLite app.db ■ ■■■■ Initialize Version 1 ■ ■■■■
Unstructured (PDF/DOCX/TXT) ■■■■ Extract plaintext content ■■■■ Chunk text by paragraphs ■■■■
Compute embeddings (all-MiniLM-L6-v2) ■■■■ Create FAISS Index & docs database ■■■■ Register
Workspace

5.2 Interactive Data Cleaning Panel

[React Panel API call] ■■■> [cleaning_routes.py] ■ [Get Workspace Info] ■ [Fetch Raw Data from
Version File] ■ [Apply Selected Cleanings] ■■■■ Standardize Column Names ■■■■ Drop Duplicates &
Trim Whitespace ■■■■ Impute Values (Unknown/Median) ■■■■ Outlier IQR / Z-score Flagging ■■■■
Recover Formulas (Derived Metrics) ■ [Calculate Post-Cleaning Score] ■ [Generate Analytics
Integrity Check] (Alert if row or metric change > 1%) ■ [Save to /uploads/cleaned as CSV] ■
[Register New Version in SQLite db] ■ [Overwrite active table inside DuckDB]

5.3 Automated Query Orchestration

[Incoming Query String] ■ [check active workspace type] ■ [Quantitative Dataset Workspace?]
■■■■ Yes: [Chat History Check] ■ ■■■■ Identical Question? ■ ■ ■■■■ Return matching cache ■ ■ ■
■■■■ Minor variation? ■ ■■■■ Adjust previous SQL ■ ■■■■ Execute in DuckDB ■ ■■■■ Update answer
text ■ ■■■■ Return ■ ■■■■ No: [Document Text Workspace?] ■■■■ Run FAISS RAG Search & return

5.4 Text-to-SQL Execution with Footnote Audits

[Run Normal Generation Pipeline] ■ [CodeGenerationAgent Prompt] ■ [Generated SQL query] ■
[Verify Safety Exceptions] ■ [Execute in DuckDB] ■ [Format Group columns] (Identify missing
values in dimensions) ■ [Build Null Footnotes Stats] (Row count and revenue metric impact) ■
[Format '■■■ Unassigned' tags] (Apply Rule 1 & 2 grouping, push to bottom) ■ [Generate
Dimension Completeness %] ■ [InsightAgent Synthesis] ■ [VisualizationAgent Plotly JSON] ■
[Dual LLM Consistency Validation] (Check Question vs SQL vs Result vs Answer) ■ [Block/Accept
Mismatch Errors] ■ [Save chat into SQLite]

6. Advanced Intelligent Features

6.1 Programmatic and LLM-Based Consistency Validation

Before returning a response, the platform verifies consistency:

- **LLM Consistency Validator:** Prompting Gemini to compare the question, generated SQL, results, synthesized answer, and Plotly chart configuration to identify hallucinations or logical mismatches.
- **Programmatic Consistency Checks:**
 - Verifies that Plotly chart axes map to existing columns in the SQL result set.
 - Extracts numbers from plain-text insights and verifies that they are present in the SQL output.

6.2 Automatic Diagnostics Corrections

If consistency verification fails:

- The system checks if the dataset has undergone cleaning.
- If not, it automatically runs the cleaning agent using recommended operations (null imputation, duplicate deletion, outliers flagging).
- It updates the workspace, regenerates the SQL statement, reruns it in DuckDB, and recreates the insight and chart.

6.3 Semantic Query Cache & Schema Adaptation

To optimize API usage:

- The orchestrator checks previous workspace queries using Levenshtein distance and LLM classifications.
- **Exact match:** Returns the cached response from SQLite.
- **Minor variation:** Adapts the SQL query (e.g., updating a year or category filter), executes it, runs the insight updater, and saves the new record.

6.4 Missing Attributions Footnote Engine

If aggregations contain missing dimensions:

- **Rule 1 & Rule 2:** Replaces null/empty categories with **■ ■ Unassigned (Missing Column Name)** and pushes them to the bottom of the table.
- **Rule 3:** Appends a footnote detailing the exact row count, units, and revenue impacted by missing categories (e.g., **■ ■ Data Quality Notice: X rows (Y units / \$Z revenue) could not be attributed...**).
- **Rule 5:** Appends a completeness score (e.g., **Dimension Completeness: X% of rows have a valid value**).

7. Multi-Format Report Exporter Engine

Defined in [export_routes.py](#), this component compiles custom exports:

- **PDF Exporter (ReportLab)**: Generates multi-page PDFs using a two-pass canvas (**NumberedCanvas**) to dynamically calculate and render "Page X of Y" footers, cover pages, and text formatting.
- **PowerPoint Presentation Exporter (python-pptx)**: Generates 16:9 slide decks with cover slides, dataset profiling tables, and query insights.
- **Word Document Exporter (python-docx)**: Compiles formatted reports with headings, tables, code blocks, and embedded charts.
- **Report Wizard Compiler**: Combines text summaries, SQL listings, and charts from the chat history into a single download.

8. Database Schemas & Storage Mappings

8.1 SQLite DB System Schema

Table: workspaces

Stores active workspace details, data types, and summaries.

```
CREATE TABLE workspaces( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, -- Display name of workspace
file_name TEXT, -- Uploaded file name
file_path TEXT, -- Path to local raw file
table_name TEXT, -- Assigned DuckDB table name
schema_json TEXT, -- Column name schema and datatype mappings
metadata_json TEXT, -- Row count, columns array, and numeric categories
cleaning_report_json TEXT, -- Generated cleaning statistics report
cleaning_approved INTEGER DEFAULT 0, -- Status flag (1 = Approved, 0 = Pending)
document_text TEXT, -- Extracted text for RAG indexing
document_tables INTEGER, -- Number of tables parsed
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

Table: chat_history

Maintains records of query interactions.

```
CREATE TABLE chat_history( id INTEGER PRIMARY KEY AUTOINCREMENT, workspace_id INTEGER, question TEXT, answer TEXT, sql_query TEXT, -- Generated DuckDB SQL statement
result_json TEXT, -- JSON representation of query results
chart_json TEXT, -- Plotly chart configuration
JSON created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(workspace_id) REFERENCES workspaces(id) );
```

Table: chat_messages

Stores chat history for active session memory.

```
CREATE TABLE chat_messages( id INTEGER PRIMARY KEY AUTOINCREMENT, workspace_id INTEGER, role TEXT, -- 'user' or 'assistant' message
TEXT, -- Chat bubble text
sql_query TEXT, result_json TEXT, chart_json TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(workspace_id) REFERENCES workspaces(id) );
```

Table: dataset_versions

Tracks versions for data cleaning rollbacks.

```
CREATE TABLE dataset_versions( id INTEGER PRIMARY KEY AUTOINCREMENT, workspace_id INTEGER,
version_num INTEGER, -- Version number (e.g. 1, 2, 3...) file_path TEXT, -- Local file path to
version CSV table_name TEXT, -- DuckDB table identifier description TEXT, -- Details of changes
applied created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(workspace_id) REFERENCES
workspaces(id) );
```

Table: reports

Lists generated reports.

```
CREATE TABLE reports( id INTEGER PRIMARY KEY AUTOINCREMENT, workspace_id INTEGER, title TEXT,
file_path TEXT, -- Path to local file for serving file_type TEXT, -- File extension (pdf, pptx,
docx, html, xlsx) created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(workspace_id)
REFERENCES workspaces(id) );
```

Table: schedules

Stores automated background report schedules.

```
CREATE TABLE schedules( id INTEGER PRIMARY KEY AUTOINCREMENT, workspace_id INTEGER, name TEXT,
-- Name of schedule query TEXT, -- SQL query statement frequency TEXT, -- 'daily', 'weekly',
'monthly', or 'cron' cron_expr TEXT, -- CRON schedule string next_run TIMESTAMP, -- Calculated
next execution time created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(workspace_id)
REFERENCES workspaces(id) );
```

9. FastAPI REST API Endpoint Dictionary

Method	Endpoint Path	Tags	Description
POST	/upload/	Upload	Handles upload of tabular/document files. Returns schema and metadata.
POST	/chat/	Chat	Entry point for natural language questions. Triggers SQL or RAG pipeline.
GET	/workspaces/	Workspaces	Lists all workspaces.
GET	/workspaces/active	Workspaces	Returns the currently active workspace.
POST	/workspaces/{id}/active	Workspaces	Switches the active workspace.
GET	/workspaces/{id}/profile	Profiling	Generates diagnostic profiles and recommendations.

Method	Endpoint Path	Tags	Description
POST	/workspaces/{id}/clean	Cleaning	Runs data cleaning based on recommended actions.
POST	/workspaces/{id}/approve_cleaning	Cleaning	Confirms cleaning changes.
GET	/workspaces/{id}/versions	Cleaning	Lists cleaning versions available for rollback.
POST	/workspaces/{id}/versions/{v}/rollback	Cleaning	Rolls back the workspace to a specified version.
POST	/schedule-report	Schedules	Registers a cron schedule.
GET	/reports/scheduled	Schedules	Lists registered schedules.
DELETE	/reports/scheduled/{id}	Schedules	Deletes a schedule.
GET	/reports	Schedules	Lists generated reports.
POST	/chart/generate	Charts	Generates Plotly JSON configurations.
GET	/workspaces/{id}/export/pdf	Export	Generates a ReportLab PDF report.
GET	/workspaces/{id}/export/pptx	Export	Generates a PowerPoint presentation.
GET	/workspaces/{id}/export/docx	Export	Generates a Word document.
POST	/workspaces/{id}/export/wizard	Export	Compiles a report containing selected chats.

10. Frontend Client Console & UI Architecture

The client interface is built as a single-page application (SPA) focused on interactive data visualization.

10.1 Key UI Modules

- [App.jsx](#): The main application layout. It coordinates workspace selections, triggers cleaning workflows, renders data profile charts, and displays chat panels.
- [UploadPanel.jsx](#): A drag-and-drop file uploader supporting various file types.
- [WorkspaceList.jsx](#): Sidebar listing files, indexing states, and current data quality scores.
- [ChatPanel.jsx](#): Chat interface displaying assistant responses, SQL code windows, and data tables.

- [ChartView.jsx](#): Component rendering Plotly.js charts with zoom, pan, and download actions.
- [ResultTable.jsx](#): Paginated table component for displaying SQL query outputs.

10.2 Styling Themes

Custom styling is configured in [index.css](#), utilizing Tailwind CSS and CSS variables to define a responsive dark mode layout. It uses glassmorphic styling, smooth scroll transitions, and custom scrollbars to highlight interactive elements.

11. Production Setup, Deployment & Dockerization

11.1 Local Installation Checklist

1. **Clone/Copy Workspace**: Navigate to the directory:

```
cd ai-data-analytics-agent
```

2. **Environment Setup**: Create a **.env** file in the root directory:

```
GOOGLE_API_KEY=your_google_gemini_api_key SQLITE_DB_PATH=database/app.db
```

3. **Backend Dependencies**: Set up a virtual environment and install dependencies:

```
python -m venv .venv # Windows: .venv\Scripts\Activate.ps1 # macOS/Linux: source  
.venv/bin/activate pip install --upgrade pip pip install -r requirements.txt
```

4. **Launch Backend**: Run the FastAPI server:

```
python main.py
```

The backend server will run on **http://127.0.0.1:8000**. You can access the API documentation at **http://127.0.0.1:8000/docs**.

5. **Frontend Dependencies**: Open a new terminal, navigate to the frontend directory, install modules, and run the development server:

```
cd frontend npm install npm run dev
```

The React console will run on **http://localhost:5173**.

11.2 Dockerization

The platform is containerized using [Dockerfile](#) for multi-stage builds.

- **Stage 1**: Installs Node.js dependencies, builds Vite static assets, and outputs them to **/frontend/dist**.
- **Stage 2**: Sets up a Python 3.11 environment, installs **requirements.txt** dependencies, copies static assets to serve via FastAPI, and exposes port **8000**.

To build and run the Docker container:

```
# Build the image docker build -t ai-data-analytics-agent . # Run the container docker run -p 8000:8000 -e GOOGLE_API_KEY=your_api_key ai-data-analytics-agent
```

The application will be accessible at **http://localhost:8000**.

12. Architectural Verification & Quality Control

The platform implements automated validations to verify system stability:

1. **Analytical Integrity Checks:** During data cleaning, the system monitors changes in row counts and key metrics (e.g., total sales, cogs, unit counts). If variations exceed 1%, it issues a warning flag.
2. **Query Safety Checks:** Prior to database execution, queries are inspected for forbidden keywords to prevent unauthorized modifications.
3. **Consistency Auditing:** It compares user questions, generated SQL, query outputs, and plain-text summaries to prevent hallucinations in insights and charts.